

LOS Hackathon

AI Agentic Engineering Hackathon

Build a personal AI assistant using an AI agent

Overview

Build a personal AI assistant that acts as an extension of your mind. It should feel like a conversation — you talk to it naturally and it captures, organizes, and acts on your life for you.

The product must be built using an AI agent.

The agent is your primary development tool — it writes code, scaffolds the project, makes decisions, and builds the product on your behalf. You direct it, it builds it.

The product itself can be anything — a web app, mobile app, CLI tool, API, whatever form best serves your vision. You are free to choose.

Participation

- **Solo or team** — you may participate individually or in a team of up to 2 people
 - **All team members** are expected to contribute
 - **One submission** per person or team
 - **Development begins** on the **9th of May 2026** and ends on the **16th of May 2026**.
-

AI Agent Constraints

These constraints apply to the AI agent you use to build your product:

- **CLI-based** — the agent must operate via the command line. This is intentional. Working within a CLI environment is a core skill in agentic engineering and a key learning objective of this hackathon
 - **Free tier only** — the agent must use only free tier API access (Gemini, GitHub Copilot, etc.)
-

Product Requirements

Your product should aim to deliver the following:

MUST HAVE

- **Note taking** — capture thoughts, ideas, and meeting notes conversationally
- **Task management** — create, update, track, and complete tasks
- **Natural conversation** — feels like talking to an assistant, not filling out a form

SHOULD HAVE

- **Reminders** — set and trigger reminders at the right time
- **Retrieval** — ask questions about your own notes and task history intelligently
- **Daily briefing** — a single command that summarizes pending tasks, upcoming meetings, and recent notes

STRETCH GOALS

- **Calendar integration** — book, reschedule, and query meetings
- **Proactive suggestions** — agent surfaces things unprompted without being asked
- **Cross-linking** — agent notices connections between your notes, tasks, and calendar automatically

Agentic Paradigms

Below is a list of agentic paradigms for your inspiration. These are techniques and patterns you are encouraged to use when working with your CLI agent during development. You are not required to use all of them — they describe how you build, not what you build. The more sophisticated your use of these paradigms, the higher your score.

Beginner

- **Planning** — using your agent in planning mode to break down tasks before executing
- **Skills** — giving your agent discrete callable abilities such as reading a file, searching the web, or calling an API
- **Commands** — structuring your agent interactions as explicit instructions rather than open-ended conversation

Intermediate

- Multiple skills chained together
- Structured commands
- Basic conversation memory and context management
- Error handling and retries
- Token awareness — agent does not blindly dump everything into the prompt

Advanced

- Sub-agents — delegating specific tasks to a secondary agent
- Streaming output
- Multi-step autonomous task execution
- Token efficiency — trimming, summarizing, or compressing context intelligently
- Context window management — knowing what to keep, what to drop, what to summarize

Expert

- Multi-agent orchestration — multiple specialized agents collaborating
- Persistent memory across sessions
- Token conservation strategies — caching, prompt reuse, selective retrieval, summarizing old turns

Deliverables

Each participant or team must present a live demo. The demo is 10 minutes per person or team.

Half 1 — Product Demo

Show your product working live. Demonstrate it being used naturally and conversationally.

Half 2 — Agent Walkthrough

Walk through how you used your CLI agent to build the product. Cover:

- Which CLI agent you used and why
- How you structured your prompts and commands
- Which agentic paradigms you used during development and how
- How you managed your token budget throughout the build
- What worked, what did not, and what you would do differently

This second half is just as important as the first. A great product built naively scores lower than a good product built with sophisticated agentic engineering.

Judging

Submissions are evaluated by a panel of AI judges — Claude, Gemini, and ChatGPT.

Criteria	Weight
Product completeness — how much of the requirements were delivered	25%
Demo quality — does it work live, is it compelling to watch	25%
Agentic sophistication — how skillfully the developer used agentic paradigms during the build	25%
Token efficiency — how intelligently the agent managed its budget	15%
Creativity and innovation — going beyond the brief in an interesting way	10%

Good luck. Build something you would actually use.